Construction and Implementation of Denotators in Rubato Composer

Munawar Khan

27 January 2023

Abstract

This article discusses the construction and implementation of musical denotators in Rubato Composer using concepts from Mathematical Music Theory and category theory. The notions of forms, denotators, and diaffine transformations are introduced, followed by the implementation of the Hunzai folk melody *Baig-e-Dani* in Rubato Composer. The work demonstrates how categorical structures provide a rigorous framework for representing and transforming musical objects.

Keywords: Mathematical Music Theory, Category Theory, Rubato Composer, Denotators, Forms, Diaffine Transformations.

Contents

1 Introduction **2**

2 Categorical Concepts **2**
2.1 Category of modules with diaffine transformations 3
2.2 Limits and Colimits 3

3 Architecture of Concepts **4**
3.1 Pure Architecture 4
3.2 Architecture with Primitives 5
3.3 Forms and Denotators 6

4 Implementation in Rubato Software **7**
4.1 Objective 7
4.2 Denotators 7

5 Conclusion and future work **10**
5.1 Musical Output 11

1 Introduction

Music can be described mathematically because musical structures exhibit well-defined relationships between frequency, rhythm, and harmony. Relationships such as octaves and fifths can be described using numerical ratios in many tuning systems, while musical pieces, in their simplest form, can be viewed as sequences of musical notes that follow certain structural relationships, such as melodic, rhythmic, or probabilistic patterns. These mathematical properties make music suitable for formal representation and computational analysis.

Rubato Composer is a software system developed within the framework of Mathematical Music Theory by Guerino Mazzola and Gérard Milmeister [3]. Unlike traditional music software, it represents musical objects using mathematical structures called forms and denotators, allowing musical information to be modeled and transformed in a rigorous way.

The objective of this report is to demonstrate the construction and implementation of denotators in Rubato Composer through the implementation of the folk melody Baige-Dani[1].

2 Categorical Concepts

Category theory[2] provides a formal framework for describing relationships between mathematical structures and is used in Mathematical Music Theory to model and transform complex musical objects.

A category is a mathematical structure consisting of a collection of objects and the morphisms that connect them. A morphism is a mathematical map that describes a relationship between two objects. In this project, morphisms are used to describe transformations between mathematical structures such as modules.

Definition 2.1. *A category $\mathscr{C}$ consists of:*

1. *a collection of objects* $\mathrm{ob}(\mathscr{C})$,
2. *for every pair of objects A, B, a collection of morphisms $Hom_{\mathscr{C}}(A, B)$,*
3. *a composition operation $g \circ f \in Hom_{\mathscr{C}}(A, C)$ for $f \in Hom_{\mathscr{C}}(A, B)$ and $g \in Hom_{\mathscr{C}}(B, C)$,*
4. *an identity morphism $1_A \in Hom_{\mathscr{C}}(A, A)$ for every object A.*

The composition is associative and satisfies the identity laws:

$$h \circ (g \circ f) = (h \circ g) \circ f, \qquad f \circ 1_A = f = 1_B \circ f.$$

Categories are mathematical structures, and functors provide a way to relate them while preserving their structure.

Definition 2.2. *Let $\mathscr{A}$ and $\mathscr{B}$ be categories. A functor $F : \mathscr{A} \to \mathscr{B}$ assigns:*

1. *each object $A \in \mathscr{A}$ to an object $F(A) \in \mathscr{B}$,*
2. *each morphism $f : A \to B$ to a morphism $F(f) : F(A) \to F(B)$.*

It preserves composition and identity morphisms:

$$F(g \circ f) = F(g) \circ F(f), \qquad F(1_A) = 1_{F(A)}.$$

2.1 Category of modules with diaffine transformations

Lemma 2.3. *If φ:$R \longrightarrow S$ is a ring homomorphism and N is S-Module. Define*

$$\cdot_\varphi : R \times N \longrightarrow N$$

$$(r, n) \mapsto \varphi(r) \cdot n$$

N is an R-Module under the scalar operation $\cdot_\varphi$, this module is denoted by $N_{[\varphi]}$.

Definition 2.4. *Let $\varphi : R \longrightarrow S$ be a ring homomorphism. Given an R-Module M and an S-module N. A* dilinear homomorphism *from M to N is a pair (φ:$R \longrightarrow S$, f_0:$M \longrightarrow N_{[\varphi]}$).*
For $n \in N$, let

$$T^n : N \longrightarrow N$$

$$a \mapsto a + n$$

then, $f = T^n \circ f_0$ is called a diaffine transformation.

2.1.1 Category of Modules with Diaffine Transformations

The category $\mathscr{M}$ provides the mathematical framework for modeling musical transformations in Rubato Composer.

- **Objects:** The objects are modules over rings.
- **Morphisms:** For modules M and N, $Hom_{\mathscr{M}}(M, N)$ consists of diaffine transformations of the form $f = T^n \circ f_0$, where f_0 is a dilinear homomorphism and T^n is a translation.
- **Composition:** The composite of two diaffine transformations $f = T^{n_2} \circ f_0$ and $g = T^{n_3} \circ g_0$ is also a diaffine transformation, defined as:

$$g \circ f = T^{g_0(n_2)+n_3} \circ (g_0 \circ f_0)$$

- **Category Axioms:** For any module N, the identity 1_N is a diaffine transformation. Composition in $\mathscr{M}$ is associative and satisfies the standard identity laws.

2.2 Limits and Colimits

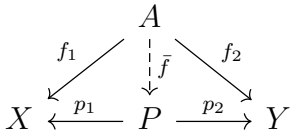
Limits and colimits provide the categorical foundation for constructing complex musical objects from simpler components.

2.2.1 Products (Conjunctions)

In Rubato Composer, the most important type of limit for musical modeling is the Product, which serves as the mathematical analog for a conceptual conjunction.

Definition 2.5. *In a category $\mathscr{C}$, a product of objects X and Y consists of an object P and projection maps $p_1 : P \to X$ and $p_2 : P \to Y$ such that for any object A and maps*

$f_1 : A \to X, f_2 : A \to Y$, *there exists a unique map* $\bar{f} : A \to P$ *making the diagram commute, as follows:*

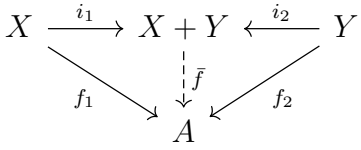


Musical Application: In this framework, a **Note** is defined as a product (limit) of its attributes: Onset, Pitch, Duration, Loudness, and Voice. If any single component is missing, the entire note object remains undefined.

2.2.2 Coproducts (Disjunctions)

The primary colimit used is the **Coproduct**, which models the choice between different musical states.

Definition 2.6. *In a category* $\mathscr{C}$*, a coproduct of objects* X *and* Y *consists of an object* $X + Y$ *and inclusion maps* $i_1 : X \to X + Y$ *and* $i_2 : Y \to X + Y$ *such that for any object* A *and maps* $f_1 : X \to A, f_2 : Y \to A$*, there exists a unique map* $\bar{f} : X + Y \to A$ *making the diagram commute, as follows:*



Musical Application: A **GeneralNote** is modeled as a coproduct (disjunction) of a **Note** and a **Rest**. This represents an "either/or" relationship where exactly one component must be given to determine the instance.

3 Architecture of Concepts

In this section, we investigate the hierarchical construction of musical concepts from their foundational mathematical structures. To understand this architecture, it is helpful to view it as a blueprint for musical objects. The system is divided into two primary parts: **pure architecture**, which defines the logical "skeleton" and rules of the system, and **architecture with primitives**, which provides the "atomic materials" (such as Onset and Pitch) used to fill that skeleton.

3.1 Pure Architecture

The pure architecture of concepts is built upon the following principles:

1. **Naming Principle:** To distinguish between mathematically identical spaces, every concept is assigned a unique name. For example, while the Euclidean space $\mathbb{R}^2$ is treated identically in pure mathematics, in this architecture, it may be named "Physical Plane" in one context and "Frequency vs. Amplitude" in another.

2. **Recursive Construction:** Concepts are built upon other concepts, which are referred to as *components*. This allows for the creation of complex musical hierarchies.

3. **Construction Types:** There are three primary ways to build a concept based on the number and relationship of its components:

   - **Selection (Power):** Method for building collections. A musical **Score** is a selection of various **Note** instances.
   - **Conjunction (Limit):** A concept determined by a fixed number of components. A **Note** is a conjunction of Onset, Pitch, Duration, and Loudness. Crucially, if one component is missing, the entire note remains undefined.
   - **Disjunction (Colimit):** A concept representing a choice between at least two components. For example, a **GeneralNote** is a disjunction of a **Note** or a **Rest**. An instance is defined only when exactly one component is provided.

3.2 Architecture with Primitives

The architecture with primitives, in the context of Rubato Composer is called *Simple*. This section introduces **Simple** concepts, which represent the basic building blocks of the architecture. Unlike composite concepts such as Limits, Colimits, and Powers, Simple concepts are not constructed from other concepts; instead, they are directly modeled by mathematical modules.

3.2.1 Simple Concepts

In the Rubato framework, basic musical attributes are modeled as follows:

- **Onset and Duration:** Modeled by the module of real numbers $\mathbb{R}$, representing continuous time-space.
- **Pitch:** Modeled by $\mathbb{Q}$ or $\mathbb{Z}$, depending on the required tuning resolution.
- **Loudness and Voice:** Typically modeled by the integers $\mathbb{Z}$.

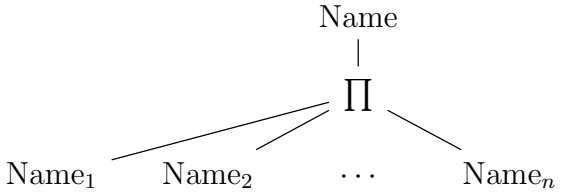
3.2.2 Diagrammatic Representation

Concepts are analyzed using tree-like notations where mathematical symbols represent construction types:

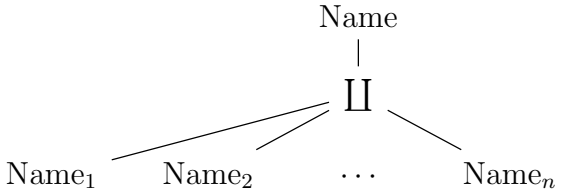
- **Limit (Π):** Represents the product, the mathematical analog of conjunction.
- **Colimit ($\amalg$):** Represents the coproduct, the mathematical analog of disjunction.
- **Power ($\{\}$):** Represents the selection method.
- **Simple:** Represents the atomic building block, drawn as the concept name placed above its modeling module (e.g., Pitch over $\mathbb{Q}$)

as shown in Figure 1.

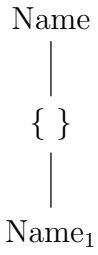
The complete hierarchy of a **Score** can be visualized as a Power type of Notes, where each Note is a Limit type of its primitive attributes (Onset, Pitch, Loudness, Duration, and Voice).



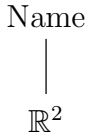
(a) Concept of type limit



(b) Concept of type colimit



(c) Concept of type power



(d) Concept of type simple on $\mathbb{R}^2$

Figure 1: Concepts.

3.3 Forms and Denotators

3.3.1 Forms

A form is a mathematical space and is formally defined as follows based on the presentation in [4]

Definition 3.1. *A form F is a quadruple $F = (N_F, T_F, C_F, I_F)$ where*

1. *N_F is the name $N(F)$ of F.*
2. *T_F is one of the following symbols:* **Simple**, **Limit**, **Colimit**, *or* **Power**. *It is called the type $T(F)$ of F.*
3. *C_F is one of the following based on the type of F:*
   (a) *Case* **Simple***: C_F is a module M.*
   (b) *Case* **Power***: C_F is a form.*
   (c) *Case* **Limit** *or* **Colimit***: C_F is a diagram D of forms.*
   It is called the coordinator $C(F)$ of F. Diagram D is a diagram of functors $Fun(F_i)$ for a family of $(F_i)_i$ of forms.

[4]

3.3.2 Denotator

Denotator is a generalized point in a form, with the following definition.

Definition 3.2. *Let M be an address. An M-addressed denotator is a triple $D = (N_D, F_D, C_D)$ where,*

1. *N_D is the name $N(D)$ of D.*

2. *F_D is the form $F(D)$ of D.*

3. *C_D is an element of $M@Fun(F(D))$ called the coordinates $C(D)$ of D.*

[5]

4 Implementation in Rubato Software

Rubettes in Rubato Composer: In Rubato Composer, a rubette is a functional processing unit that performs a specific operation on denotators or other musical data. Rubettes can be connected together to form computational networks, allowing the construction of complex musical transformations. Each rubette receives input data, applies a mathematical or musical operation, and produces an output that can be passed to subsequent rubettes.

Musical Terms: In the Rubato framework, the fundamental building block of music is the **Note**, which is defined mathematically as a mandatory conjunction or limit of five foundational attributes . These attributes consist of its starting temporal position (**Onset**), its melodic frequency (**Pitch**), and the length of the sound in continuous time (**Duration**) . Furthermore, a note is defined by its intensity (**Loudness**) and its assigned instrumental part (**Voice**) . While a **Rest** represents a period of silence determined only by its onset and duration, a **GeneralNote** represents the conceptual choice or colimit between a note and a rest . Finally, a **Score** is defined as a collection or selection (power) of individual notes that together constitute a complete musical structure

4.1 Objective

The objective of this section is to demonstrate the practical application of the **Form** and **Denotator** language through the implementation of the Hunzai folk melody *'Baig-e-Dani'* 2. This implementation serves as a concrete realization of the mathematical structures discussed in the preceding sections, illustrating how categorical morphisms can rigorously model and transform a musical motif within the Rubato Composer software framework.

4.2 Denotators

Before constructing the complete "Note" object, we must define its individual components as denotators. In this implementation, each attribute is modeled as a morphism from a 55-dimensional integer module $\mathbb{Z}^{55}$ (representing the motif's 55 events) to the appropriate mathematical module.

4.2.1 Loudness and Voice

To focus the implementation on pitch and temporal transformations, the loudness and voice attributes are kept constant. Mathematically, these are represented as simplified diaffine transformations where the linear part is zero.

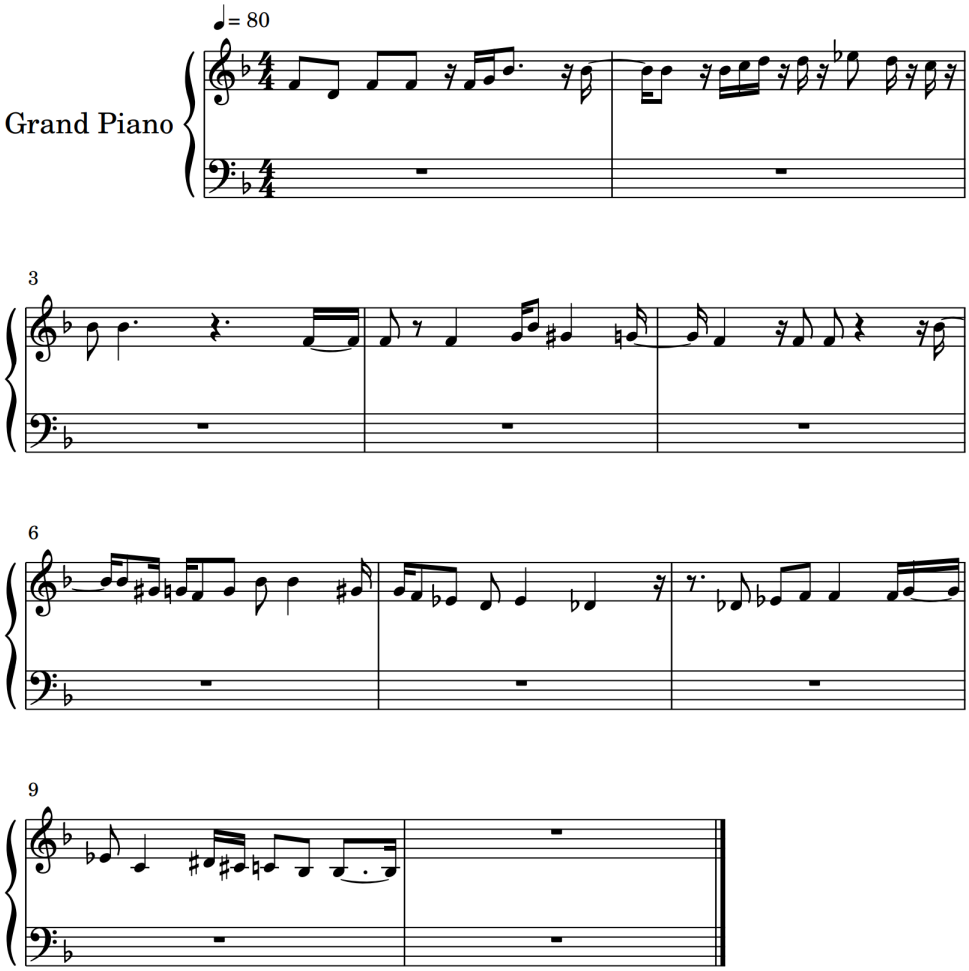


Figure 2: Baig-e-Dani

- **Loudness:** Modeled by the module morphism $m_l : \mathbb{Z}^{55} \to \mathbb{Z}$. For this project, the loudness is set to a constant MIDI value of 60:

$$x \mapsto 60$$

- **Voice:** Modeled by the module morphism $m_v : \mathbb{Z}^{55} \to \mathbb{Z}$. The voice is kept constant at 0:

$$x \mapsto 0$$

4.2.2 Pitch

The pitch attribute is modeled as a denotator within the Rubato framework using a module morphism $m_p : \mathbb{Z}^{55} \to \mathbb{Q}$. This morphism is implemented as a composition of an embedding morphism and an affine morphism, maintaining the categorical structure established in section 1:

$$m_p = m_{p,1} \circ m_{p,2}$$

1. **Embedding Morphism ($m_{p,2}$):** This map, $m_{p,2} : \mathbb{Z}^{55} \to \mathbb{Q}^{55}$, serves to embed the discrete integer representations of the motif's 55 events into a rational vector space.

2. **Affine Morphism ($m_{p,1}$):** This map, $m_{p,1} : \mathbb{Q}^{55} \to \mathbb{Q}$, defines the specific melodic contour of the *'Baig-e-Dani'* motif:

$$x \mapsto (-3, 0, 0, 0, 2, 5, 5, 5, 5, 7, 9, 9, 10, 9, 7, 5, 5, 0, 0, 0, 2, 5, 3, 2, 0, 0, 0, 5, 5, 3, 2, 0,$$
$$2, 5, 5, 3, 2, 0, -2, -3, -2, -4, -4, -2, 0, 0, 0, 2, -2, -5, -2, -4, -5, -7, -7) \cdot x + 66$$

Following Definition 2.4, the **translation part** of this affine morphism is **66**, which determines the pitch of the starting note. In the context of this implementation, MIDI number 66 corresponds to the note **F#4**. All subsequent notes in the motif are determined relative to this initial note through the linear part of the morphism.

4.2.3 Onset

The onset attribute is modeled as a denotator within the Rubato framework using a module morphism $m_o : \mathbb{Z}^{55} \to \mathbb{R}$. This morphism is implemented as a composition of an embedding morphism and an affine morphism:

$$m_o = m_{o,1} \circ m_{o,2}$$

1. **Embedding Morphism ($m_{o,2}$):** This map, $m_{o,2} : \mathbb{Z}^{55} \to \mathbb{R}^{55}$, embeds the discrete integer representations of the motif's 55 events into a real vector space.

2. **Affine Morphism ($m_{o,1}$):** This map, $m_{o,1} : \mathbb{R}^{55} \to \mathbb{R}$, determines the temporal positions of the motif events:

$$x \mapsto (1, 2, 3, 4.5, 5, 5.5, 7.5, 8.5, 10, 10.5, 11, 12, 13, 14, 15, 16, 17, 23, 24, 26,$$
$$27.75, 28.5, 29.25, 31.5, 33, 35, 36, 40, 41, 42, 42.5, 43, 44, 45, 46, 48, 48.5,$$
$$49, 50, 51, 52, 54, 58, 59, 60, 61, 62.5, 63, 64, 65, 67, 67.5, 68, 69, 70) \cdot x$$

The affine morphism has a translation part of **0**, which determines the onset of the first note. Therefore, in this implementation, the motif begins at time zero.

4.2.4 Duration

The duration attribute is modeled as a denotator within the Rubato framework using a module morphism $m_d : \mathbb{Z}^{55} \to \mathbb{R}$. This morphism is implemented as a composition of an embedding morphism and an affine morphism:

$$m_d = m_{d,1} \circ m_{d,2}$$

1. **Embedding Morphism ($m_{d,2}$):** This map, $m_{d,2} : \mathbb{Z}^{55} \to \mathbb{R}^{55}$, embeds the discrete integer representations of the motif's 55 events into a real vector space.

2. **Affine Morphism ($m_{d,1}$):** This map, $m_{d,1} : \mathbb{R}^{55} \to \mathbb{R}$, defines the duration values of the individual notes:

$$x \mapsto (0, 0, 0, -\tfrac{1}{2}, -\tfrac{1}{2}, \tfrac{1}{2}, 0, 0, -\tfrac{1}{2}, -\tfrac{1}{2}, -\tfrac{1}{2}, -\tfrac{1}{2}, 0, -\tfrac{1}{2}, -\tfrac{1}{2},$$
$$0, 2, 0, 0, 0, 0, 0, 4, 1, 1, 0, 0, 0, 0, -\tfrac{1}{2}, -\tfrac{1}{2}, 0, 0, 0, 1, -\tfrac{1}{2},$$
$$-\tfrac{1}{2}, 0, 0, 0, 1, 1, 0, 0, 0, \tfrac{1}{2}, -\tfrac{1}{2}, 0, 0, 1, -\tfrac{1}{2}, -\tfrac{1}{2}, 0, 0, 1) \cdot x + 1$$

Following Definition 2.4, the **translation part** of this affine morphism is **1**, which determines the duration of the starting note. In this implementation, the initial note has a duration value of 1.

4.2.5 Note

Now, we create the note denotator, with the denotators: onset, loudness, pitch, voice, and duration as its components.

4.2.6 Network

Finally, we connect the source rubette with Module Map rubette and do an inversion at $F\#_4$. then, we evaluate the original note and the inverted note using the AddressEval rubette. Now we use the Set rubette and take the union of the evaluated notes and play the result using the ScorePlay rubette. Figure 3 shows the constructed rubette network used for the transformation, while Figure 4 displays the resulting score generated by the ScorePlay rubette.

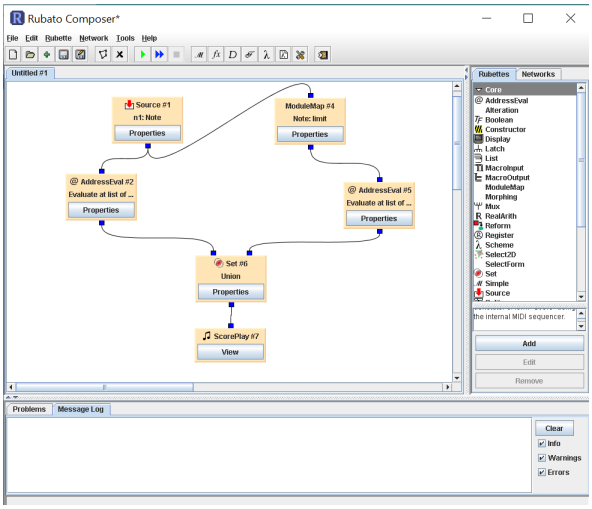


Figure 3: Network of rubettes

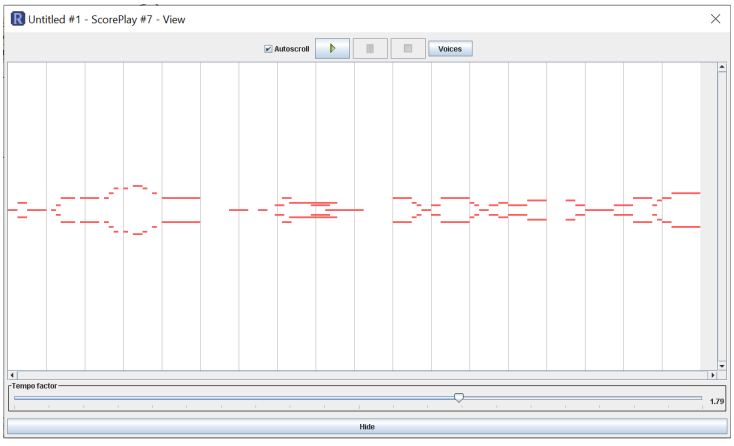


Figure 4: ScorePlay

5 Conclusion and future work

The project demonstrated that categorical structures such as forms, denotators, and di-affine transformations provide a systematic framework for representing and transforming

musical structures. The implementation of *Baig-e-Dani* showed how an abstract mathematical description can be converted into an executable musical object within Rubato Composer.

5.1 Musical Output

The final implementation of the *Baig-e-Dani* melody generated in Rubato Composer can be accessed through the following video demonstration:

Demonstration of the implemented melody

This project can be extended by introducing more dynamic morphisms. For example, non-constant morphisms for the loudness and voice denotators could be used to incorporate greater musical expression and variation. Furthermore, the framework can be extended to represent multiple musical instruments and more complex rhythmic structures.

Another interesting direction for future work is the implementation of Indian classical music pieces in Rubato Composer. In this project, pitch was represented using discrete points; however, many classical musical traditions involve continuous pitch variations. One possible approach would be to connect discrete pitch values using polynomial functions, allowing the representation of a continuous spectrum of frequencies rather than only discrete pitch points.

Acknowledgements

I would like to thank Dr. Mariana Montiel for her supervision, guidance, and support throughout this project.

References

[1] Sherbaz Ali Khan and Azeem Hunzai. Baig-e-dani.

[2] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, 2nd edition, 1998.

[3] Guerino Mazzola and collaborators. Rubato composer, 2023. Software environment for Mathematical Music Theory.

[4] Guerino Mazzola, Stefan Göller, Stefan Müller, and Shlomo Dubnov. The topos of music: Geometric logic of concepts, theory, and performance. *The Mathematical Intelligencer*, 27, 09 2005.

[5] Gérard Milmeister. *The Rubato Composer Music Software*. Springer, Berlin, Heidelberg, 2009.